

Treinamento de ML no Ryzen 7

03/12/2026 18:27:56

Como vai funcionar a parte de ML do meu projeto? Quero saber também o porquê,...

Thought process: Desvendou funcionamento, motivação e processo de treinamento...

Eu já não encontro esse ML treinado pronto disponível? Outra coisa, eu tenho que...

Thought process: Avaliou disponibilidade de modelos pré-treinados e arquitetou...

Perfeito crie um documento com todo esse planejamento e documentação de...

Thought process: Estruturando documentação abrangente e plano competitivo de...

Uma duvida que fiquei sobre o treinamento, você acha que faz sentido eu subir...

Thought process: Validou estratégia de APIs vulneráveis e agentes IA para...

Perfeito, como o shannon foi projetado para testes whitebox, eu passo o código do...

Thought process: Pensando em como o Shannon funciona para testes whitebox e s...

Então primeiro fazemos análises com estes agentes atacando e nosso gateway em...

Thought process: Validou estratégia de teste em dois modos sequenciais. The user...

👤 Como vai funcionar a parte de ML do meu projeto? Quero saber também o porquê, em que e como eu irei fazer o treinamento de ml no meu Ryzen 7

3/10/2026, 6:17:29 AM

Thought process: Desvendou funcionamento, motivação e processo de treinamento ML.

The user wants to understand the ML part of their Dorsal project - specifically how it works, why they're training on the Ryzen 7, what they'll train, and how the training process works.

ML no Dorsal — Como, Por quê e No Ryzen

O que o ML resolve aqui

As regras OWASP (SQLi, XSS, BOLA...) detectam ataques **conhecidos** por padrão. O ML detecta **anomalias comportamentais** — coisas que nenhuma regex consegue pegar, como:

- Um endpoint que normalmente recebe 10 req/min de repente recebe 800
- Um usuário que sempre acessa `/api/orders/{id}` com IDs sequenciais (enumeração)
- Um payload que tem tamanho, estrutura e timing fora do padrão histórico daquele endpoint
- Um user-agent que aparece às 3h da manhã com entropia de parâmetros atípica

O modelo não sabe "isso é SQLi". Ele sabe "isso é diferente do normal" — e isso é complementar às regras.

O Algoritmo: Isolation Forest

O plano usa **Isolation Forest** do scikit-learn, e a escolha é muito inteligente para o seu contexto:

Por quê Isolation Forest e não redes neurais?

Critério	Isolation Forest	Neural Network
Dados necessários	Poucos (tráfego normal)	Muitos (exemplos de ataque)
Treinamento	Minutos no Ryzen	Horas/dias
Inferência	~0.1ms	~1-5ms
Explicabilidade	Razoável	Baixa
Deploy	ONNX leve (~50KB)	Modelo pesado
Tipo	Unsupervised	Precisa de labels

O ponto mais importante: você **não tem labels**. Você não sabe quais requests dos seus beta testers são ataques. Isolation Forest é **unsupervised** — aprende o que é "normal" e pontua desvios.

As Features (o que você vai alimentar no modelo)

Por endpoint (ex: `POST /api/v1/users/search`):

```
features = {
  # Volume
  "requests_per_minute": float, # taxa atual vs média histórica
  "requests_per_minute_ratio": float, # atual / baseline dos últimos 7 dias

  # Payload
  "body_size_bytes": float, # tamanho do body
```

```

    "body_size_z_score":          float, # desvio padrão do
tamanho normal
    "param_count":                int,    # número de parâmetros
    "param_entropy":              float,  # aleatoriedade dos
valores (strings random = alta)

    # Temporal
    "hour_of_day":                int,     # 0-23 (ataques bot =
madrugada)
    "is_weekend":                 bool,
    "inter_request_ms":           float,   # tempo entre requests
do mesmo IP

    # Response
    "status_code":                int,     # 200, 401, 403, 500...
    "response_size_bytes":        float,
    "response_size_ratio":        float,   # atual / baseline

    # Source
    "user_agent_entropy":         float,   # user-agent aleatório
= bot
    "ip_request_count_24h":        int,     # quantas requests esse
IP fez
    "is_new_ip":                  bool,     # IP nunca visto antes?

    # Sequência (para BOLA)
    "resource_id_sequential":      float,   # IDs estão
incrementando?
    "unique_resource_ids_per_min": int,     # quantos IDs distintos
em 1 min
}

```

O Pipeline Completo de Treinamento no Ryzen

Gateway do cliente

|

| telemetria assíncrona (gRPC, best-effort)

▼

Control Plane (OCI)

|

| dados anonimizados

▼

RYZEN 7 5800X

1. Coleta dados do ClickHouse
(janela: últimos 7-14 dias,
tráfego normal dos gateways)

2. Feature Engineering
(pandas / polars)

3. Treina Isolation Forest
(scikit-learn, ~10min)

4. Valida contra dados holdout
(verifica FP rate < 3%)

5. Exporta para ONNX
(skl2onnx, ~50-200KB)

6. Embarca no container
gateway/dorsal/ml/models/

|

| nova versão do container



Docker Hub → clientes fazem pull

O Código em si (como vai ficar)

Treinamento (`scripts/ml-train/train.py` — roda no Ryzen):

```
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
import joblib

# 1. Puxa dados do ClickHouse (tráfego normal anonimizado)
df = fetch_normal_traffic(days=14)

# 2. Feature engineering
features = engineer_features(df)

# 3. Treina — contamination=0.01 = espera 1% de anomalias nos
dados de treino
scaler = StandardScaler()
X = scaler.fit_transform(features)

model = IsolationForest(
    n_estimators=100,          # 100 árvores
    contamination=0.01,       # 1% esperado de outliers
    max_samples=256,          # eficiência
    random_state=42,
    n_jobs=-1                  # usa todos os 16 threads do 5800x
)
```



```

model.fit(X)

# 4. Valida FP rate
fp_rate = validate_false_positives(model, scaler)
assert fp_rate < 0.03, f"FP rate muito alto: {fp_rate}"

# 5. Exporta ONNX
initial_type = [("float_input", FloatTensorType([None,
X.shape[1]]))]
onnx_model = convert_sklearn(model, initial_types=initial_type)

with open("gateway/dorsal/ml/models/anomaly_v2.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

print(f"Modelo exportado. FP rate: {fp_rate:.2%}")

```

Inferência (`gateway/dorsal/ml/anomaly.py` — roda no container do cliente):

```

import onnxruntime as rt
import numpy as np

class AnomalyDetector:
    def __init__(self, model_path="models/anomaly_v1.onnx"):
        self.session = rt.InferenceSession(model_path)
        # ONNX carrega em ~50ms na inicialização, inferência é
        ~0.1ms

    async def score(self, features: dict) -> float:
        """Retorna anomaly score: -1 a 0 (quanto mais negativo,
        mais anômalo)"""
        x = np.array([[features[f] for f in FEATURE_ORDER]],
dtype=np.float32)

        # Inferência assíncrona — não bloqueia o proxy
        result = self.session.run(None, {"float_input": x})

```

```
score = float(result[0][0]) # isolation forest score

return score

def is_anomaly(self, score: float, threshold: float = -0.1) -
> bool:
    return score < threshold
```

O Behavioral Baseline (complemento ao Isolation Forest)

Além do modelo global, cada endpoint aprende seu próprio "normal" nos primeiros 7 dias:

```
Endpoint: GET /api/v1/orders/{id}
├─ Baseline aprendido:
│   └─ req/min: média 45, desvio 12
│   └─ body_size: média 0 bytes (GET)
│   └─ response_size: média 1.2KB
│   └─ IPs únicos/hora: média 23
└─ Alerta quando:
    └─ req/min > média + 3σ (spike)
    └─ response_size < 100B (dado truncado = possível bloqueio)
    └─ IPs únicos/hora < 2 (um IP varrendo sozinho = BOLA)
```

Por que o Ryzen 7 5800x é perfeito para isso

O treinamento **não precisa de GPU** porque Isolation Forest é CPU-bound e paraleliza muito bem:

- **16 threads** (8 cores + HT) = treina em paralelo com `n_jobs=-1`

- **32GB RAM** = carrega meses de dados de telemetria sem swap
- **NVMe rápido** = I/O não vira gargalo na leitura de dados
- **Roda offline** = zero custo de cloud, zero dependência de OCI para treinar
- **Você controla o schedule** = pode treinar toda semana com dados novos sem custo

O processo completo leva **menos de 15 minutos** no 5800x, incluindo feature engineering de 14 dias de dados de dezenas de gateways.

O Ciclo de Melhoria Contínua

Semana 1-7: Modelo v1 (treinado em dados sintéticos)
Semana 8: Primeiros beta testers geram tráfego real
Semana 10: Treina modelo v2 com dados reais anonimizados
Semana 12: FP rate medido: se > 3%, ajusta contamination e threshold
Mês 3+: Retrain semanal automático no Ryzen via cron
Se FP rate aumentou: modelo anterior é mantido (rollback automático)

O resumo prático: o Ryzen é seu **laboratório de ML offline gratuito**. O modelo vai de scikit-learn para um arquivo ONNX de ~100KB que embarca direto no container Docker do cliente — ele roda inferência em tempo real sem jamais fazer chamada de rede.

3/10/2026, 6:18:24 AM

👤 Eu já não encontro esse ML treinado pronto disponível? Outra coisa, eu tenho que obrigatóriamente treinar com as minhas orgs, meus clientes ou posso usar outros dados para treinar? Qual melhor lógica pra isso?

3/10/2026, 6:21:28 AM

Thought process: Avaliou disponibilidade de modelos pré-treinados e arquitetou estratégia híbrida de treinamento.

The user is asking two good questions:

1. Can they find pre-trained ML models for this use case instead of training from scratch?
2. Do they need to train with their own clients' data, or can they use other data sources? What's the best logic?

Let me think through this carefully.

Question 1: Pre-trained models available?

Yes, there are some options:

- **ONNX Model Zoo** - has some security-related models but nothing specific for API anomaly detection
- **Hugging Face** - some anomaly detection models but not API traffic specific
- **CrowdSec** - open source, has behavioral models

- **Falco** - runtime security but different domain
- **ModSecurity CRS** - rule-based, not ML
- **Snyk/OWASP** - mostly rule-based

The reality is there's no good pre-trained model for API traffic anomaly detection you can just drop in, because "normal" traffic is highly specific to each application—a fintech's patterns look

completely different from an e-commerce API. That said, there are pre-trained models for specific attack patterns like SQLi/XSS classifiers on Hugging Face trained on SecLists, though these are more limited in scope.

For training data, I'm thinking a phased approach makes sense: start with public datasets like UNSW-NB15, CIC-IDS2017/2018, and KDD Cup 99, plus synthetic data generated from attack tools and SecLists payloads for the MVP phase. Then move to opt-in anonymized telemetry from beta clients once we have initial traction.

The architecture I'm settling on is a hybrid model—a global baseline trained on public and synthetic data to catch known attack patterns, paired with per-tenant behavioral models that learn what's normal for each specific API using their first week of traffic without any cross-customer data sharing. This mirrors how Datadog handles APM anomaly detection, combining global patterns with personalized baselines.

Duas perguntas excelentes e muito práticas. Vou ser direto.

Pergunta 1: Existe modelo pré-treinado disponível?

Resposta curta: para detecção de ataques específicos (SQLi, XSS), sim. Para anomalia comportamental de tráfego de API, não — e existe um motivo fundamental para isso.

O que existe pronto

Para detecção de payloads maliciosos (substitui ou complementa suas regras regex):

- **PayloadAllTheThings + SecLists** — não são modelos, são datasets públicos massivos de payloads de ataque reais. Você treina um classificador binário simples com isso em horas
- **HuggingFace** — existem alguns modelos de detecção de SQL injection e XSS, mas são pesados demais para inferência em tempo real em um proxy
- **CrowdSec behavior engine** — open source, tem modelos de comportamento mas focados em IP reputation, não em análise de payload
- **ModSecurity CRS** — rule-based, não ML, mas é o padrão da indústria e você pode usar as regras como feature engineering

Para anomalia de rede/tráfego geral:

- **UNSW-NB15** — dataset acadêmico clássico de intrusão de rede, dá pra treinar Isolation Forest com ele
- **CIC-IDS2017/2018** — Canadian Institute for Cybersecurity, tráfego de rede real com ataques rotulados

Por que não existe modelo pré-treinado para o seu caso específico

O problema é **fundamental**, não técnico:

Fintech: POST /pix/transfer → body: 2KB, 50 req/min, 99% status 200

E-commerce: POST /pix/transfer → não existe esse endpoint

O que é "normal" para uma API é completamente diferente do que é "normal" para outra. Um modelo global treinado em dados mistos seria inútil — teria FP rate > 30% na prática.

Isso é exatamente por isso que Salt Security, Traceable e Noname cobram \$70k/ano — eles passam semanas fazendo o modelo aprender o baseline específico daquele cliente antes de ativar proteção.

Pergunta 2: Como treinar sem dados de clientes? Qual a melhor lógica?

A melhor lógica é uma **arquitetura em 3 camadas** que evolui conforme seu produto cresce. Você nunca fica bloqueado esperando dados de clientes.

A Arquitetura

CAMADA 1: Global Attack Model

"Isso parece um ataque?"

Treinado em dados públicos

Estático, embarcado no container

Detecta: SQLi, XSS, path traversal, SSRF

+

CAMADA 2: Per-Tenant Behavioral Baseline

"Isso é normal para ESTA API?"

Treinado nos primeiros 7 dias de tráfego

do PRÓPRIO cliente, dentro do container dele

Dados nunca saem da infra do cliente

+

CAMADA 3: Global Anomaly Model (futuro)

"Isso é estatisticamente raro em todos os
clientes opt-in?"

Treinado em telemetria anonimizada agregada

Só ativa quando você tem dados suficientes

Fase 0 — MVP (antes de qualquer cliente)

Você treina com dados **100% públicos**. Isso já é suficiente para lançar.

Dataset para o Global Attack Model:

Fontes públicas que você usa HOJE no Ryzen

```
sources = {
  "SecLists": {
    "url": "github.com/danielmiessler/SecLists",
    "uso": "Payloads de SQLi, XSS, path traversal, SSRF",
    "tamanho": "~500k exemplos de payloads maliciosos",
    "label": 1 # positivo = ataque
  },
  "OWASP_WebGoat_traffic": {
    "url": "github.com/WebGoat/WebGoat",
    "uso": "Tráfego HTTP real de app vulnerável sendo
atacada",
    "label": 1
  },
  "Common_Crawl_API_traffic": {
    "uso": "Tráfego HTTP legítimo de APIs públicas",
    "label": 0 # negativo = tráfego normal
  },
  "UNSW_NB15": {
    "url": "research.unsw.edu.au/projects/unsw-nb15-dataset",
    "uso": "Network traffic com ataques rotulados",
    "label": "mixed"
  },
  "Synthetic_normal": {
    "uso": "Você gera tráfego normal sintético com faker +
httpx",
    "label": 0
  }
}
```

```
}  
}
```

Gerando tráfego normal sintético (você faz isso no Ryzen antes do lançamento):

```
# scripts/ml-train/generate_synthetic.py  
import faker  
import random  
  
fake = faker.Faker('pt_BR')  
  
def generate_normal_request():  
    """Simula tráfego legítimo de uma API REST típica"""  
    return {  
        "method": random.choices(  
            ["GET", "POST", "PUT", "DELETE"],  
            weights=[60, 25, 10, 5]  
        )[0],  
        "path": random.choice([  
            f"/api/v1/users/{random.randint(1, 10000)}",  
            "/api/v1/users/me",  
            f"/api/v1/orders/{fake.uuid4()}",  
            "/api/v1/products?page=1&limit=20",  
        ]),  
        "body_size": random.gauss(mu=512, sigma=200),  
        "param_count": random.randint(0, 5),  
        "param_entropy": random.gauss(mu=2.1, sigma=0.4),  
        "requests_per_minute": random.gauss(mu=45, sigma=15),  
        "hour_of_day": random.choices(  
            range(24),  
            # Distribuição realista: mais tráfego de dia  
            weights=[  
                1, 1, 1, 1, 1, 2, 4, 8, 10, 10, 10, 9, 8, 9, 10, 10, 9, 8, 7, 6, 5, 4, 3, 2  
            ]  
        )[0],  
        "status_code": random.choices(  

```

```

        [200, 201, 400, 401, 404, 500],
        weights=[70, 10, 8, 5, 5, 2]
    )[0],
    "label": 0 # normal
}

# Gera 100k exemplos normais
dataset = [generate_normal_request() for _ in range(100_000)]

```

Fase 1 — Beta Testers (7 a 14 dias de dados)

A Camada 2 (**behavioral baseline**) é treinada **dentro do container do cliente**, nos primeiros 7 dias. Isso é um diferencial competitivo enorme — você pode dizer aos clientes:

"Seus dados de tráfego nunca saem da sua infraestrutura. O modelo aprende o comportamento das suas APIs localmente."

```

# gateway/dorsal/ml/baseline.py
# Isso roda DENTRO do container do cliente

class EndpointBaseline:
    """
    Aprende o comportamento normal de cada endpoint
    nos primeiros 7 dias. Após isso, entra em modo de detecção.
    """

    LEARNING_DAYS = 7

    def __init__(self, endpoint: str):
        self.endpoint = endpoint
        self.is_learning = True
        self.observations = []

```



```

self.baseline = {}

def observe(self, metrics: dict):
    if self.is_learning:
        self.observations.append(metrics)

    # Após 7 dias com dados suficientes, calcula baseline
    if self._has_enough_data():
        self._compute_baseline()
        self.is_learning = False
        # Treina Isolation Forest LOCAL com esses dados
        self._train_local_model()

def _compute_baseline(self):
    import numpy as np
    obs = self.observations

    self.baseline = {
        "req_per_min_mean": np.mean([o["req_per_min"] for o
in obs]),
        "req_per_min_std": np.std([o["req_per_min"] for o in
obs]),
        "body_size_mean": np.mean([o["body_size"] for o in
obs]),
        "body_size_std": np.std([o["body_size"] for o in
obs]),
        # ... outras métricas
    }

def score(self, current_metrics: dict) -> float:
    """Retorna quantos desvios padrão o request atual está do
normal"""
    if self.is_learning:
        return 0.0 # ainda aprendendo, não alerta

```

```
z_scores = []
for metric, value in current_metrics.items():
    mean = self.baseline.get(f"{metric}_mean", 0)
    std = self.baseline.get(f"{metric}_std", 1)
    if std > 0:
        z_scores.append(abs(value - mean) / std)

return max(z_scores) if z_scores else 0.0
```

Fase 2 — Escala (opt-in para melhorar o modelo global)

Quando você tiver 20+ clientes, pode oferecer opt-in de telemetria **agregada e anonimizada**:

Dashboard > Settings > Privacy

[] Contribuir com telemetria agregada anonimizada para melhorar a detecção global do Dorsal.

Compartilhamos APENAS: distribuições estatísticas de tráfego (médias, desvios). NUNCA payloads, IPs reais ou dados de negócio.

[Saiba mais] [Ativar — eu ganho modelo melhorado]

Com isso você treina o modelo da Camada 3 no Ryzen com dados reais de dezenas de ambientes, sem violar privacidade.

Resumo da Estratégia

HOJE (zero clientes):

- ✓ Global Attack Model → SecLists + dados sintéticos
- ✓ Já funciona para SQLi, XSS, path traversal
- ✓ Embarca **no** container v1

SEMANA 8 (primeiros beta testers):

- ✓ Behavioral Baseline → aprende nos 7 primeiros dias
- ✓ Dados ficam **NO** container **do** cliente
- ✓ Você não vê nada, cliente confia mais

MÊS 3+ (20+ clientes opt-in):

- ✓ Retrain semanal **no** Ryzen com telemetria agregada
- ✓ Modelo global fica progressivamente melhor
- ✓ Vira um moat competitivo real

A beleza dessa arquitetura é que **você lança com dados sintéticos, entrega valor real desde o dia 1 com o baseline local, e o modelo melhora sozinho conforme cresce** — sem depender de nenhum cliente para dar o primeiro passo.

3/10/2026, 6:22:39 AM

👤 Perfeito crie um documento com todo esse planejamento e documentação de treinamento, passo a passo para eu executar em cada fase, não esqueça de incluir a payloadAllTheThings e todas as melhores fontes iniciais. Assim como, a arquitetura de tudo para eu apresentar o funcionamento da ferramenta para os clientes, os pontos chave...fica na sua infra...funciona assim etc. Por fim, você consegue me garantir um plano de treinamento incrivelmente eficiente pra me colocar no nível dos meus concorrentes como salt etc e até superar eles?

3/10/2026, 7:26:12 AM

Thought process: Estruturando documentação abrangente e plano competitivo de treinamento.

The user wants a comprehensive document covering:

1. ML training plan - step by step for each phase
2. Data sources (PayloadAllTheThings, etc.)
3. Architecture documentation for client presentations
4. Key selling points
5. A training plan to compete with/beat Salt Security etc.

Let me check the SKILL.md for docx first, then create a comprehensive document.

View: Reading docx skill

Thought process: Orchestrated comprehensive ML documentation com arquitetura, treinamento e estratégia competitiva.

Now I'll create a comprehensive Word document covering all the ML training plan, architecture, and competitive strategy. This will be a large document. Let me write the JavaScript code for it.

Vou criar um documento Word completo e profissional. Deixa eu montar isso agora.

Bash Tool: Setup npm docx

Create File: Dorsal ML training plan document

Bash Tool: Generate the docx

Thought process: Identificou e corrigiu referência de cor hexadecimal malformada.

I need to fix the color reference. There's a `C.gold + "FF"` which appended FF to the color making it 8 digits. Let me fix that.

Edit: Fix invalid hex color

Bash Tool: Generate document

Edit: Fix shading color with alpha

Bash Tool: Generate document

Bash Tool: Validate document

Present File: /mnt/user-data/outputs/dorsal_ml_arquitetura.docx

Documento criado com 548 parágrafos e 6 seções completas. O que está dentro:

Seção 1 — Visão Executiva do ML: tabela das 3 camadas (quando cada uma ativa, onde roda, o que precisa para treinar) + o argumento de privacidade para usar na venda.

Seção 2 — Fontes de Dados: guia completo de PayloadAllTheThings (o que baixar de cada pasta, o comando git clone), SecLists, UNSW-NB15, CIC-IDS, Juice Shop, DVWA, ModSecurity CRS, NVD/CVE — e como gerar tráfego normal sintético com Faker e Locust para contrapeso.

Seção 3 — Arquitetura para Apresentar: o diagrama Data Plane vs Control Plane, fluxo de cada request com timing por etapa, tabela completa das 20 features do ML com o que cada uma detecta, e respostas prontas para as perguntas difíceis dos clientes.

Seção 4 — Plano de Treinamento Fase a Fase: código real para cada fase — build_dataset.py, RandomForest Camada 1, critérios de aceitação do modelo, Isolation Forest local Camada 2, pipeline de retreino semanal no Ryzen com cron.

Seção 5 — Estratégia Competitiva: tabela comparativa honesta Dorsal vs Salt vs Traceable vs Noname, os 5 diferenciais que criam moat real, e o roadmap de 12 meses para chegar à paridade técnica deles (e onde você já os supera no dia 1).

Seção 6 — Checklists: ações numeradas para cada fase, sem ambiguidade.

File: /mnt/user-data/outputs/dorsal_ml_arquitetura.docx

3/10/2026, 7:32:05 AM

👤 Uma duvida que fiquei sobre o treinamento, você acha que faz sentido eu subir algumas APIs vulneraveis, estas conhecidas OWASP ou similar (me indique, inclusive), dentro de uma ORG de teste quando eu subir pra produção e atacar ela faz sentido pra ajudar na coleta de dados para treinamento? Como sou especialista em offsec pensei em fazer isto mas n sei se faz sentido, outra duvida é se eu utilizar agentes de AI como strix (<https://github.com/usestrix/strix>) e shannon (<https://github.com/KeygraphHQ/shannon>) iria ajudar a coletar dados de tecnicas amplamente utilizadas, modernas, assim como, treinar contra ataques de agentes de AI (pode ter diferencial se fizer sentido). Queria saber tambem se faz sentido usar APIs de exemplo vulneraveis ou outras aplicacoes de exemplo que nao estao (quais), eu subiria tudo nessa org de testes, entao seria uima simulacao de uma empresa que tem diversas apis e aplicacoes no ambiente

3/10/2026, 2:13:24 PM

Thought process: Validou estratégia de APIs vulneráveis e agentes IA para treinamento defensivo.

The user is asking three things:

1. Does it make sense to spin up vulnerable APIs (OWASP-style) in a test org, attack them to collect training data?
2. Would AI agents like Strix and Shannon help collect data on modern attack techniques and AI agent attacks?
3. Does it make sense to use vulnerable example APIs/apps to simulate a real company environment?

Let me check the Strix and Shannon GitHub links mentally - Strix is an offensive security AI agent, Shannon appears to be a graph-based AI agent framework. I should search to verify these.

Let me think through the answers:

1. Yes, attacking vulnerable apps in a test org is EXCELLENT for training data. This is actually the gold standard approach. Apps

to recommend:

- OWASP Juice Shop (Node.js, REST API)
- WebGoat (Java)
- DVWA (PHP)
- crAPI (Completely Ridiculous API - specifically designed for API security testing)
- VAmPI (Vulnerable API)
- Pixi (OWASP API Security Top 10)
- Damn Vulnerable GraphQL Application (DVGA)
- vulnapi
- API Security Project

2. AI agents as attackers - this is very interesting. Using Strix and Shannon to systematically attack these vulnerable applications could generate rich datasets on both traditional attack patterns and how AI agents approach security testing differently than humans.

3. A full company environment with multiple interconnected applications makes sense for realistic threat modeling. Let me verify what Strix and Shannon actually are.

Ótimas perguntas — e a sua intuição de offsec está certa. Deixa eu verificar as ferramentas que você mencionou primeiro.

Web Fetch: Fetched: GitHub - usestrix/strix: Open-source AI hackers to find and fix your app's vulnerabilities.

Thought process

The page content was truncated. Let me also check Shannon.

Web Fetch: Fetched: GitHub - KeygraphHQ/shannon: Fully autonomous AI hacker to find actual exploits in your web apps. Shannon has achieved a 96.15% success rate on the hint-free, source-aware XBOW Benchmark.

Thought process: Validou estratégia de teste com APIs vulneráveis e agentes IA.

Now I have a clear picture. Let me give a comprehensive answer:

1. Yes, attacking vulnerable APIs in a test org is absolutely the gold standard for training data collection. This is what I'd recommend.

2. Strix = open-source AI hackers to find and fix vulnerabilities
Shannon = fully autonomous AI hacker with 96.15% success rate on XBOW benchmark, 10.6k stars - this is a serious tool.

Both are legitimate and would be extremely valuable for generating AI-agent-specific attack signatures.

Let me give a thorough answer about the test org setup, the apps to use, and the AI agent strategy.

Perfeito — agora tenho o contexto completo. Resposta direta e detalhada:

Questão 1: Org de teste + atacar APIs vulneráveis = faz sentido?

Faz todo o sentido. É o gold standard. Na verdade é o que as empresas sérias de ML em segurança fazem internamente — elas chamam de "red team data generation". Você tem

uma vantagem enorme aqui por ser offsec: não vai apenas rodar ferramentas automatizadas, vai atacar com a criatividade e o contexto de um humano.

A diferença entre dados sintéticos (SecLists) e dados gerados por você atacando ativamente uma app vulnerável é enorme:

- SecLists tem os **payloads**, mas não tem o **contexto de sessão** — sequência de requests, timing, headers reais de um browser/tool real
- Quando você ataca o Juice Shop manualmente ou com burp, você gera **fluxos de ataque completos**: autenticação → reconnaissance → exploit → exfiltration. Isso é o que acontece em produção
- Você consegue rotular com precisão: "esse fluxo de 47 requests é um ataque BOLA confirmado que funcionou"

Os apps vulneráveis para subir na org de teste

Aqui está a lista ideal para simular uma empresa real com diversas APIs e tecnologias:

APIs REST/JSON (core do Dorsal):

App	Stack	OWASP Coverage	Por que incluir
crAPI (Completely Ridiculous API)	Node + Python	API Top 10 completo	Projetado especificamente para OWASP API Top 10. O melhor.
OWASP Juice Shop	Node.js	API Top 10 + Web	O mais completo, tem REST API + GraphQL + desafios
VAmPI	Python/Flask	BOLA, Mass Assignment, Injection	Minimalista, fácil de instrumentar
Pixi	Node.js	Broken Auth, Data Exposure	Focado em auth e PII
WebGoat	Java/Spring	IDOR, Injection, Auth	Cobre casos enterprise (Java APIs)

GraphQL (para depois do MVP):

App	Por que incluir
DVGA (Damn Vulnerable GraphQL App)	Introspection abuse, query depth, batch attacks
HackTheBox GraphQL labs	Ataques reais registrados

Apps completas (não só APIs — para tráfego legítimo):

App	Por que incluir
DVWA	Tráfego web legítimo misturado com ataques
Metasploitable 3	Simula servidor enterprise vulnerável
VulnHub machines	Variedade de stacks

Aplicações legítimas para contrapeso (label 0):

App	Por que incluir
Gitea (self-hosted)	REST API real de produto em uso, tráfego normal
Nextcloud	API de arquivo, autenticação real
Mattermost	Websocket + REST, padrão de uso real de empresa

Questão 2: Strix e Shannon como geradores de dados

Shannon é especialmente sério — 96.15% de success rate no XBOW Benchmark hint-free com 10.6k stars. Isso não é toy project. É um hacker autônomo real.

Por que usar AI agents como atacantes é um diferencial genuíno:

Os ataques gerados por agentes de AI têm assinaturas comportamentais completamente diferentes de ataques humanos ou de ferramentas tradicionais:

Ataque humano (burp manual):

POST /api/v1/users/1 → 200

POST /api/v1/users/2 → 200

POST /api/v1/users/3 → 200 (enumeração óbvia, inter-request ~800ms)

Ataque de ferramenta (sqlmap):

POST /api/v1/login body: user=admin'-- (payload óbvio no campo errado)

user-agent: sqlmap/1.7.8#stable

inter-request: 50ms exato

Ataque Shannon/Strix:

GET /api/v1/docs → 200 (reconnaissance de schema)

POST /api/v1/auth/login (credencial válida criada antes)

GET /api/v1/users/me → extrai user_id=4821

GET /api/v1/users/4820 → testa BOLA com ID -1

GET /api/v1/users/4819 → confirma enumeração

PATCH /api/v1/users/4819 → tenta privilege escalation

inter-request: variável (200ms a 3s), user-agent real, session válida

O Shannon com 96% de success rate está fazendo exploit chains que requerem raciocínio multi-step. Isso gera dados que nenhum dataset público tem. **Se você treinar o Dorsal para detectar esse padrão, vai ser o único produto no mercado que detecta AI agents como vetor de ataque** — e isso está se tornando um problema real à medida que mais empresas usam AI agents com acesso a APIs.

Como instrumentar isso praticamente:

Você captura o tráfego do Shannon atacando o crAPI

e gera uma feature especial: agent_attack_pattern

```
features_ai_agent = {
```

```
"session_coherence_score": float,  
# AI agents mantêm sessão coerente vs tools que não  
autenticam  
  
"reconnaissance_before_exploit": bool,  
# AI agents fazem GET /docs, /openapi.json antes de atacar  
  
"multi_step_chain": bool,  
# mais de 3 endpoints diferentes em sequência lógica  
  
"timing_variance": float,  
# AI agents têm timing variável e mais "humano" que  
ferramentas  
  
"error_adaptation": bool,  
# AI agents mudam estratégia após 403/401, ferramentas não  
  
"valid_session_used": bool,  
# Shannon se autentica antes de explorar  
}
```

A Org de Teste — Como Estruturar

A simulação de empresa é a ideia certa. Aqui está como montar:

```
org-teste-dorsal/  
|  
├─ infra/  
|   ├─ docker-compose.yml    # sobe tudo junto  
|   └─ nginx.conf            # roteamento entre apps  
|  
└─ apis-vulneraveis/
```

```

|   ├── crapi/                # API principal alvo
|   ├── vampi/                # API Flask simples
|   ├── juice-shop/          # Node + GraphQL
|   └── webgoat/              # Java enterprise
|
|── apps-legitimas/           # geram tráfego normal (label 0)
|   ├── gitea/                # dev tools
|   ├── mattermost/          # comunicação
|   └── nextcloud/            # storage
|
|── gateway-dorsal/           # o próprio Dorsal na frente de
TUDO
|   └── docker-compose.yml
|
|── atacantes/
|   ├── shannon/              # AI agent autônomo
|   ├── strix/                # AI agent offsec
|   ├── burp-scripts/         # seus scripts manuais
|   └── locust-normal/         # tráfego legítimo sintético
|
|── coleta/
|   ├── capture.py            # lê logs do Dorsal gateway
|   ├── labeler.py            # rotula por janela de tempo
(attack on/off)
|   └── export_dataset.py      # gera parquet para treino no Ryzen

```

O fluxo de coleta:

O Dorsal fica na frente de tudo como gateway. Você liga o modo de captura, roda um ciclo de ataque, para, roda tráfego normal. O gateway já extrai as features — você só precisa adicionar o label baseado em qual fase estava rodando:

```

# coleta/labeler.py
import pandas as pd

```

```
def label_by_timeline(logs_df, attack_windows):  
    """  
    attack_windows = [  
        ("2026-03-10 14:00", "2026-03-10 14:30", "shannon_bola"),  
        ("2026-03-10 15:00", "2026-03-10 15:45", "strix_sqli"),  
        ("2026-03-10 16:00", "2026-03-10 18:00", "normal"),  
    ]  
    """  
    logs_df["label"] = 0  
    logs_df["attack_type"] = "normal"  
  
    for start, end, attack_type in attack_windows:  
        mask = (logs_df.timestamp >= start) & (logs_df.timestamp  
<= end)  
        logs_df.loc[mask, "label"] = 0 if attack_type == "normal"  
    else 1  
        logs_df.loc[mask, "attack_type"] = attack_type  
  
    return logs_df
```

O Diferencial Competitivo Real

Juntando tudo, o que você está construindo é:

Salt Security dataset: tráfego de clientes **enterprise** (não controlado, parcialmente rotulado)

Dorsal dataset:

✓ PayloadAllTheThings → 15k payloads estáticos (mesmos que todos têm)

✓ SecLists → wordlists **conhecidas** (mesmos que todos têm)

- ✓ Você atacando manual → exploit chains reais com contexto offsec
- ✓ Shannon attacks → AI agent exploitation [patterns](#) (NINGUÉM tem isso)
- ✓ Strix attacks → AI agent + fix [loop](#) (padrão de ataque diferente)
- ✓ Tráfego legítimo real → Gitea + Mattermost + Nextcloud (não sintético)
- ✓ Labels precisos → você controlou quando cada ataque rodou

Você chega em meses no que Salt levou anos para construir — porque está gerando dados controlados e rotulados com precisão, não dependendo de tráfego de clientes que chega lentamente e mal rotulado.

E o "Dorsal detecta ataques de AI agents" vira um feature page inteiro quando AI agents se proliferarem como vetor de ataque — o que já está começando a acontecer.

3/10/2026, 2:14:47 PM

👤 Perfeito, como o shannon foi projetado para testes whitebox, eu passo o código do repo, faz alguma diferença ou consigo testar normalmente? eu tenho que passar o repo mas ele ataca dinamicamente a produção se eu quiser, é isso? veja como funciona e esclareça este ponto

3/10/2026, 2:20:44 PM

Thought process: Pensando em como o Shannon funciona para testes whitebox e se passar o código do repositório faz diferença nos testes de produção.

Let me fetch more details about Shannon to understand how it works.

Web Fetch: Failed to fetch:

<https://raw.githubusercontent.com/KeygraphHQ/shannon/main/README.md>

Web Search: shannon KeygraphHQ how it works whitebox blackbox source code required

GitHub - KeygraphHQ/shannon: Shannon Lite is a fully autonomous AI pentester for web apps and APIs. 96.15% (100/104 exploits) on a hint-free, source-aware variant of the XBOW benchmark. · GitHub github.com

shannon/README.md at main · KeygraphHQ/shannon github.com

Getting Started | KeygraphHQ/shannon | DeepWiki deepwiki.com

shannon by KeygraphHQ - SourcePulse sourcepulse.org

Completely Automate Penetration Testing with AI - Pinggy pinggy.io

Shannon and the Rise of "Proof-by-Exploitation" AI Pentesting | by Barbaros Emre Alagöz | Medium medium.com

Shannon: AI Pentesting Tool That Autonomously Identifies and Exploits Code Vulnerabilities gbhackers.com

GitHub - Steake/shannon-uncontained: Shannon-Uncontained. A Docker-free, black-box-first fork that treats AI as a fallible witness rather than an oracle: treats uncertainty as a first class citizen via EQBSL epistemic bookkeeping, and agentic recon/analysis/synthesis tuned for "paste url -> pwn box -> ??? -> profit" github.com

shannon/SHANNON-PRO.md at main · KeygraphHQ/shannon github.com

shannon/CLAUDE.md at main · KeygraphHQ/shannon github.com

Thought process: Consolidou entendimento completo sobre funcionamento do Shannon.

Now I have a complete picture. Let me explain clearly how Shannon works.

Agora tenho o quadro completo. Resposta direta:

Como o Shannon funciona de fato

Shannon é estritamente white-box e requer acesso ao código-fonte. Seus agentes de exploração ativamente modificam os alvos, tornando-o inadequado para ambientes de produção.

O comando de execução deixa claro o modelo:

```
./shannon start URL=https://your-app.com REPO=my-repo
```

Você passa **dois parâmetros obrigatórios**: a URL da app rodando E o repositório com o código. O REPO deve ser uma pasta dentro de `./repos/`, não um path absoluto — você

clona ou cria um symlink do seu repo lá primeiro.

O que ele faz com o código (não é só varredura)

Shannon emula um pentester humano usando arquitetura multi-agent. Combina análise profunda de código white-box com exploração dinâmica black-box em quatro fases distintas: reconhecimento, análise de vulnerabilidades, exploração e relatório. Agentes especializados analisam o codebase e a infraestrutura, depois executam ataques ao vivo via browser para confirmar vulnerabilidades com provas de conceito concretas.

O detalhe importante é **como ele usa o código**: Shannon Pro transforma o codebase em um Code Property Graph (CPG) que combina AST, control flow graph e program dependence graph em uma estrutura unificada. Para cada tipo de vulnerabilidade, o sistema identifica sources (onde dados não confiáveis entram) e sinks (onde podem causar dano). Pattern matching determinístico estabelece uma baseline, e então um agente AI analisa o codebase para descobrir sources e sinks que padrões genéricos não capturam, incluindo handlers de input customizados e padrões específicos do framework.

Ou seja: ele lê o código para **saber onde atacar com precisão cirúrgica**, depois ataca a instância rodando dinamicamente para provar que funciona.

Para o seu caso: faz sentido e como usar

Sim, faz todo o sentido — e você tem uma vantagem específica: os apps vulneráveis (crAPI, Juice Shop, VAmPI) **têm o código-fonte público**. Você passa o repo deles para o Shannon e a URL da instância que você subiu. Exatamente o caso de uso que ele foi projetado para resolver.

```
# Exemplo com crAPI na sua org de teste
git clone https://github.com/OWASP/crAPI.git ./repos/crapi
```

```
./shannon start \  
  URL=http://crapi.org-teste.dorsal.local:8080 \  
  REPO=crapi \  
  WORKSPACE=crapi-audit-01
```

O Dorsal gateway fica na frente do `crapi.org-teste.dorsal.local` e captura tudo que o Shannon gera.

O problema de licença que você precisa saber

Shannon Lite é licenciado sob AGPL-3.0. Essa licença permite uso interno e modificação privada, mas exige open-source das modificações se o software for oferecido como serviço público ou gerenciado. Para geração de dados de treino internamente isso não é problema — você não está redistribuindo o Shannon.

Uma alternativa blackbox que você pode usar junto

Existe um fork relevante que você descobriu: Shannon-Uncontained é um fork Docker-free, black-box-first, projetado para "paste url → pwn box → profit" — ou seja, funciona **sem passar o código-fonte**. Isso é útil para testar apps que você não tem o código (por exemplo, APIs internas simuladas que você mesmo escreveu para a org de teste sem querer dar o código ao atacante).

Como isso se encaixa na sua estratégia de dados

O fluxo ideal para a org de teste fica assim:

App vulnerável (crAPI, Juice Shop...)

| código-fonte



Shannon Lite — ataca —▶ instância rodando



Dorsal gateway captura



labels por janela de tempo



dataset com exploit chains

reais geradas por AI agent

O dado que isso gera — exploit chains multi-step com sessão coerente, timing variável e reconnaissance antes do ataque — é o que diferencia seu dataset de treinamento de qualquer coisa disponível publicamente, e o que vai permitir ao Dorsal detectar ataques de AI agents, algo que Salt e Traceable hoje simplesmente não fazem porque esse vetor era irrelevante quando eles construíram seus modelos.

3/10/2026, 2:21:26 PM

👤 Então primeiro fazemos análises com estes agentes atacando e nosso gateway em modo monitor....depois de coletar e treinar o modelo, fazemos um outro ataque coordenado de agentes mas desta vez com gateway inline para ver se vai bloquear ou se tiver no monitor alertar. é isso?

3/10/2026, 2:24:28 PM

Thought process: Validou estratégia de teste em dois modos sequenciais.

The user is asking about the validation workflow - first monitor mode to collect data and train, then inline mode to test if the trained model blocks the attacks. They want confirmation this is the right approach.

Exatamente. Você acabou de descrever o ciclo padrão de ML em segurança — **collect** → **train** → **validate** → **iterate**. E a sua intuição de usar os mesmos agentes nas duas fases é o que torna o teste válido. Deixa eu detalhar como executar isso de forma rigorosa:

O Ciclo Completo

FASE **A** — COLETA (gateway **em** modo MONITOR)

- |
- |— Shannon + Strix atacam livremente
- |— **Tráfego** legítimo (Locust) **em** paralelo
- |— Dorsal vê tudo, não bloqueia nada
- |— Gera dataset rotulado por janela de tempo



FASE **B** — TREINO (Ryzen offline)

|

- └─ Feature engineering no dataset coletado
- └─ Treina RandomForest (ataques conhecidos)
- └─ Treina **Isolation** Forest (anomalia)
- └─ Valida métricas: precision > 90%, FP < 5%
- └─ Exporta ONNX → embarca no container

|



FASE C – VALIDAÇÃO (gateway em modo INLINE)

- |
- └─ MESMO dataset de agentes, MESMOS targets
- └─ Dorsal agora bloqueia ou alerta
- └─ Mede: detection rate, FP rate, latência
- └─ Identifica o que passou (falso negativo)

|



FASE D – ITERATE

- |
- └─ Analisa o que escapou
- └─ Adiciona features ou ajusta threshold
- └─ Volta para Fase A com novos ataques

Um detalhe crítico: os dois ataques não podem ser idênticos

Aqui está o risco que destrói a validade do teste se você não tomar cuidado:

Se você rodar o Shannon exatamente com o mesmo repo, mesma URL, mesma sessão — ele vai gerar **os mesmos exploit chains**. O modelo vai ter "memorizado" aqueles payloads específicos e vai parecer que detecta tudo, mas na prática você mediu overfitting, não generalização.

A forma correta é separar em dois conjuntos:

```

# Durante a Fase A, você roda N sessões de ataque
# e divide ANTES de treinar

sessoes_ataque = [
    "shannon_crapi_run_01",    # segunda-feira
    "shannon_crapi_run_02",    # terça
    "shannon_juice_run_01",
    "strix_vampi_run_01",
    "manual_bola_run_01",      # você atacando manualmente
    "manual_sqli_run_02",
    ...
]

# 70% vai para treino
treino = sessoes_ataque[:int(len(sessoes_ataque) * 0.7)]

# 30% fica BLOQUEADO – nunca visto pelo modelo
holdout = sessoes_ataque[int(len(sessoes_ataque) * 0.7):]

# A Fase C usa APENAS o holdout para validar
# Se o modelo detecta o holdout bem → generaliza de verdade

```

E na Fase C, você idealmente roda o Shannon **uma vez nova**, com um workspace diferente — ele vai gerar variações nos payloads e na sequência de ataques que o modelo nunca viu.

Como configurar o gateway nos dois modos

```

# gateway/dorsal/config.py

class GatewayMode(Enum):
    MONITOR = "monitor"    # vê tudo, loga, não bloqueia

```



```
ALERT    = "alert"      # bloqueia acima do threshold, mas
avisa antes

INLINE   = "inline"     # bloqueia silenciosamente acima do
threshold

# Controlado por env var – zero downtime para trocar
DORSAL_MODE = GatewayMode(os.getenv("DORSAL_MODE", "monitor"))
DORSAL_BLOCK_THRESHOLD =
float(os.getenv("DORSAL_BLOCK_THRESHOLD", "70"))
```

```
# Fase A – monitor puro
DORSAL_MODE=monitor docker compose up

# Fase C – inline com threshold conservador primeiro
DORSAL_MODE=inline DORSAL_BLOCK_THRESHOLD=80 docker compose up

# Fase C – inline mais agressivo depois de validar FP rate
DORSAL_MODE=inline DORSAL_BLOCK_THRESHOLD=65 docker compose up
```

O que medir na Fase C

Três métricas que importam, nesta ordem de prioridade:

1. **FALSE POSITIVE RATE** ← mais importante para adoção
"Quanto tráfego legítimo o Dorsal bloqueou?"
Meta: < 3% em inline, < 1% em produção **real**
Mede: requests Locust (legítimos) que receberam **403**
2. **DETECTION RATE** ← o que você quer maximizar
"De todos os ataques do Shannon, quantos foram detectados?"
Meta: > 90% para ataques conhecidos, > 70% para AI agent
chains

Mede: requests Shannon que geraram alerta ou foram bloqueados

3. LATÊNCIA P99 ← diferencial competitivo

"O gateway adicionou quanto de latência?"

Meta: < 5ms

Mede: `timestamp` entrada vs `timestamp` forwarded ao upstream

```
# scripts/validate/measure_phase_c.py
```

```
def calculate_metrics(logs_df):
    legitimate = logs_df[logs_df["label"] == 0]
    attacks     = logs_df[logs_df["label"] == 1]

    fp_rate = len(legitimate[legitimate["blocked"] == True]) /
len(legitimate)
    detection_rate = len(attacks[attacks["detected"] == True]) /
len(attacks)
    p99_latency = logs_df["added_latency_ms"].quantile(0.99)

    # Breakdown por tipo de ataque – essencial para debugging
    by_type = attacks.groupby("attack_type").apply(lambda g: {
        "detection_rate": g["detected"].mean(),
        "count": len(g)
    })

    return {
        "fp_rate": fp_rate,          # < 0.03 = aprovado
        "detection_rate": detection_rate, # > 0.90 = aprovado
        "p99_latency_ms": p99_latency,    # < 5.0 = aprovado
        "by_attack_type": by_type,        # onde está falhando?
    }
```

O que fazer com os falsos negativos (o que escapou)

Os ataques que passaram pelo gateway inline **são mais valiosos que os que foram detectados**. Eles dizem exatamente onde o modelo é cego:

Shannon gerou 340 requests de ataque

Dorsal detectou 298 (87.6%)

Escaparam 42 requests

Análise dos 42:

- |— 18 eram BOLA com IDs não-sequenciais (hash UUIDs)
 - | → Feature `resource_id_sequential` não funciona com UUIDs
 - | → **FIX:** adicionar feature `"resource_id_entropy"`
 - |
- |— 14 eram SQLi com encoding duplo (URL encode + base64)
 - | → Regex da Camada 1 não descodifica antes de testar
 - | → **FIX:** adicionar normalização de encoding no pre-processor
 - |
- |— 10 eram ataques de timing (blind SQLi via `sleep()`)
 - | → Nenhuma feature captura tempo de resposta incomum
 - | → **FIX:** adicionar feature `response_time_z_score`

Cada falso negativo vira uma issue no GitHub com label `ml-gap`, e a próxima iteração de treino prioriza exatamente esses casos.

Cadência ideal para o ciclo completo

Semana 1: Fase A — 5 dias de coleta
(Shannon + Strix + manual + Locust rodando em paralelo)

Semana 2: Fase B — treino no Ryzen

(1-2 dias de feature engineering + treino + validação offline)

Semana 2: Fase C – validação inline
(2-3 dias com threshold conservador → agressivo)

Semana 3: Análise dos gaps → volta para Fase A
com ataques específicos para os blind spots encontrados

Depois de 3-4 ciclos você tem um modelo que:

- Viu variações suficientes dos mesmos ataques para generalizar
- Tem blind spots documentados e em tratamento
- Tem métricas publicáveis com metodologia transparente

Isso é o que vira o blog post técnico que derruba Salt na comparação pública.

3/10/2026, 2:25:18 PM